5. Остаточные сверточные нейронные сети. Batch-нормализация и drop-out.

Цель работы - реализовать и исследовать сверточную нейронную сеть с остаточными соединениями (residual CNN) для задачи классификации изображений на выборке CIFAR-10 с использованием вычислений на GPU. В рамках работы требуется реализовать несколько вариантов сверточных блоков:

1. Базовые сверточные блоки из предыдущего задания
2. Остаточные блоки с соединениями напрямую (skip-connection)
3. Bottleneck-блоки с использованием свёрток 1×1.

Дополнительно необходимо использовать слои пакетной нормализации (BatchNorm) и dropout, а также подобрать набор гиперпараметров, обеспечивающий наилучшее качество классификации. Для сравнения требуется сопоставить результаты обычной CNN (без остаточных блоков) и остаточной/ bottleneck-архитектуры на тех же данных.

Задания:

1. Реализация архитектуры CNN с различными типами блоков, BatchNorm и Dropout.
2. Подготовка выборки CIFAR-10 и обучение моделей на GPU.
3. Ручной подбор гиперпараметров (7–10 вариантов) и сравнение качества на валидационной и тестовой выборках.

Ссылка на исходный код лабораторной работы:
https://github.com/F1ameX/Modern-Methods-of-Deep-Machine-Learning/blob/main/5_residual_neural_network/5_residual_neural_network.ipynb

Ход работы

Реализация и обучение сети с ручным подбором гиперпараметров

В первой части работы была реализована универсальная архитектура ResNetLikeCNN, позволяющая собирать сеть из разных типов блоков:

1. Обычный сверточный блок (из прошлой лабораторной работы)
2. Остаточный блок (ResidualBlock) с прямым соединением:

$y = F(x) + skip(x)$

где F(x) реализовано как две свертки с BatchNorm и ReLU, а skip(x) - тождественное отображение либо 1×1 Conv при изменении числа каналов/stride.

```python
class ResidualBlock(nn.Module):
    def __init__(
        self,
        input_channels: int,
        output_channels: int,
        *,
        conv_kernel_size: int = 3,
        conv_stride: int = 1,
        conv_padding: int | None = None,
        use_batchnorm: bool = True,
        dropout: float = 0.0,
    ):
        super().__init__()

        if conv_padding is None:
            conv_padding = conv_kernel_size // 2

        self.conv1 = nn.Conv2d(
            input_channels,
            output_channels,
            kernel_size=conv_kernel_size,
            stride=conv_stride,
            padding=conv_padding,
            bias=not use_batchnorm,
        )
        self.bn1 = nn.BatchNorm2d(output_channels) if use_batchnorm else nn.Identity()

        self.conv2 = nn.Conv2d(
            output_channels,
            output_channels,
            kernel_size=conv_kernel_size,
            stride=1,
            padding=conv_padding,
            bias=not use_batchnorm,
        )
        self.bn2 = nn.BatchNorm2d(output_channels) if use_batchnorm else nn.Identity()

        if (input_channels != output_channels) or (conv_stride != 1):
            self.skip = nn.Sequential(
                nn.Conv2d(input_channels, output_channels, kernel_size=1, stride=conv_stride, bias=False),
                nn.BatchNorm2d(output_channels) if use_batchnorm else nn.Identity(),
            )
        else:
            self.skip = nn.Identity()

        self.activation = nn.ReLU(inplace=True)
        self.dropout = nn.Dropout2d(dropout) if dropout > 0 else nn.Identity()
```

```python
def forward(self, x: torch.Tensor) -> torch.Tensor:
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.activation(out)

    out = self.conv2(out)
    out = self.bn2(out)

    out = out + self.skip(x)
    out = self.activation(out)
    out = self.dropout(out)
    return out
```

Рисунок 1 - Код реализации остаточного блока

3. Bottleneck-блоки вида 1x1 → 3x3 → 1x1 + skip-connection. Такой блок уменьшает вычисления за счет сжатия числа каналов в середине, при этом сохраняя возможность строить более глубокие сети.

```python
class BottleneckBlock(nn.Module):
    def __init__(
        self,
        input_channels: int,
        output_channels: int,
        *,
        bottleneck_channels: int | None = None,
        conv_kernel_size: int = 3,
        conv_stride: int = 1,
        conv_padding: int | None = None,
        use_batchnorm: bool = True,
        dropout: float = 0.0,
    ):
        super().__init__()

        if conv_padding is None:
            conv_padding = conv_kernel_size // 2

        if bottleneck_channels is None:
            bottleneck_channels = max(8, output_channels // 4)

        self.conv1 = nn.Conv2d(input_channels, bottleneck_channels, kernel_size=1, stride=1, padding=0, bias=not use_batchnorm)
        self.bn1 = nn.BatchNorm2d(bottleneck_channels) if use_batchnorm else nn.Identity()

        self.conv2 = nn.Conv2d(
            bottleneck_channels,
            bottleneck_channels,
            kernel_size=conv_kernel_size,
            stride=conv_stride,
            padding=conv_padding,
            bias=not use_batchnorm,
        )
        self.bn2 = nn.BatchNorm2d(bottleneck_channels) if use_batchnorm else nn.Identity()

        self.conv3 = nn.Conv2d(bottleneck_channels, output_channels, kernel_size=1, stride=1, padding=0, bias=not use_batchnorm)
        self.bn3 = nn.BatchNorm2d(output_channels) if use_batchnorm else nn.Identity()

        if (input_channels != output_channels) or (conv_stride != 1):
            self.skip = nn.Sequential(
                nn.Conv2d(input_channels, output_channels, kernel_size=1, stride=conv_stride, bias=False),
                nn.BatchNorm2d(output_channels) if use_batchnorm else nn.Identity(),
            )
        else:
            self.skip = nn.Identity()

        self.activation = nn.ReLU(inplace=True)
        self.dropout = nn.Dropout2d(dropout) if dropout > 0 else nn.Identity()
```

```python
def forward(self, x: torch.Tensor) -> torch.Tensor:
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.activation(out)

    out = self.conv2(out)
    out = self.bn2(out)
    out = self.activation(out)

    out = self.conv3(out)
    out = self.bn3(out)

    out = out + self.skip(x)
    out = self.activation(out)
    out = self.dropout(out)
    return out
```

Рисунок 2 - Код реализации BottleNeck-блока

После реализации блоков была собрана итоговая модель ResNetLikeCNN, где тип архитектуры выбирается параметром architecture $\in$ {conv, res, bottle}, а размерность каналов контролируется параметрами числа блоков и базового числа каналов. Для классификации использовалась схема Global Average Pooling → Linear.

```python
class ResNetLikeCNN(nn.Module):
    def __init__(
        self,
        *,
        input_channels: int = 3,
        architecture: str = "res",
        conv_variant: str = "a",
        n_blocks: int = 3,
        base_channels: int = 32,
        channels_multiplier: int = 2,
        use_batchnorm: bool = True,
        dropout: float = 0.0,
        num_classes: int = 10,

        conv_kernel_size: int = 3,
        conv_stride: int = 1,
        conv_padding: int | None = None,

        pool_kernel_size: int = 2,
        pool_stride: int = 2,
    ):
        super().__init__()
        assert architecture in ("conv", "res", "bottle")
        assert conv_variant in ("a", "b")
        assert n_blocks >= 1

        layers = []
        channels = base_channels

        layers.extend([
            nn.Conv2d(input_channels, channels, kernel_size=3, stride=1, padding=1, bias=not use_batchnorm),
            nn.BatchNorm2d(channels) if use_batchnorm else nn.Identity(),
            nn.ReLU(inplace=True),
        ])

        input_channels = channels
```

```python
for block_id in range(n_blocks):
    stride_here = 2 if (architecture in ("res", "bottle") and block_id > 0 and block_id % 2 == 0) else 1

    if architecture == "conv":
        layers.append(
            ConvBlock(
                input_channels=input_channels,
                output_channels=channels,
                variant=conv_variant,
                conv_kernel_size=conv_kernel_size,
                conv_stride=conv_stride,
                conv_padding=conv_padding,
                use_batchnorm=use_batchnorm,
                pool_kernel_size=pool_kernel_size,
                pool_stride=pool_stride,
                dropout=dropout,
            )
        )
        input_channels = channels
        channels *= channels_multiplier

    elif architecture == "res":
        layers.append(
            ResidualBlock(
                input_channels=input_channels,
                output_channels=channels,
                conv_kernel_size=conv_kernel_size,
                conv_stride=stride_here,
                conv_padding=conv_padding,
                use_batchnorm=use_batchnorm,
                dropout=dropout,
            )
        )
        input_channels = channels
        if stride_here == 2:
            channels *= channels_multiplier

    else:  # "bottle"
        layers.append(
            BottleneckBlock(
                input_channels=input_channels,
                output_channels=channels,
                conv_kernel_size=conv_kernel_size,
                conv_stride=stride_here,
                conv_padding=conv_padding,
                use_batchnorm=use_batchnorm,
                dropout=dropout,
            )
        )
        input_channels = channels
        if stride_here == 2:
            channels *= channels_multiplier

self.features = nn.Sequential(*layers)
self.gap = nn.AdaptiveAvgPool2d((1, 1))
self.classifier = nn.Linear(input_channels, num_classes)
```

```python
  def forward(self, x: torch.Tensor) -> torch.Tensor:
      x = self.features(x)
      x = self.gap(x).flatten(1)
      logits = self.classifier(x)
      return logits
```

Рисунок 3 - Код реализации модели ResNetLikeCNN

После была подготовлена выборка CIFAR-10: выполнено разбиение на train/val и создан DataLoader для обучения, валидации и теста. Нормализация применялась по каналам RGB. Обучение проводилось на GPU (CUDA), что существенно ускоряет вычисления для сверточных моделей.

Для сравнения архитектур был выполнен ручной перебор 8 конфигураций: три варианта обычной CNN, три варианта residual-сети и два варианта bottleneck-сети (bottle_3_blocks, bottle_5_blocks). В экспериментах варьировались тип архитектуры, число блоков, базовое число каналов, размер ядра свертки, а также параметры обучения (learning rate, weight decay, dropout и число эпох). Для каждой конфигурации модель обучалась заданное число эпох, после чего фиксировались accuracy и loss на валидационной и тестовой выборках, а также время обучения; результаты сводились в итоговую таблицу для сравнения.

	name	arch	variant	n_blocks	base_ch	k	conv_s	pool_k	pool_s	bn	dropout	lr	wd	epochs	first_ep_acc>=0.90	val_acc	test_acc	test_loss	time_sec
0	conv_a_k5	conv	a	3	32	5	1	2	2	True	0.10	0.0008	0.0001	15	None	0.8156	0.8164	0.539777	194.286217
1	conv_b_small	conv	b	3	32	3	1	2	2	True	0.10	0.0010	0.0001	15	None	0.8140	0.8133	0.566307	211.466082
2	conv_a_small	conv	a	3	32	3	1	2	2	True	0.10	0.0010	0.0001	15	None	0.7684	0.7641	0.686533	192.591618
3	res_5_blocks	res	-	5	32	3	1	2	2	True	0.15	0.0006	0.0001	15	None	0.7368	0.7376	0.732361	286.819165
4	res_3_blocks	res	-	3	32	3	1	2	2	True	0.10	0.0008	0.0001	15	None	0.7064	0.7086	0.830038	248.537976
5	res_wide	res	-	3	64	3	1	2	2	True	0.20	0.0006	0.0001	10	None	0.6974	0.6915	0.865606	209.545578
6	bottle_5_blocks	bottle	-	5	32	3	1	2	2	True	0.15	0.0006	0.0001	15	None	0.5522	0.5609	1.208602	230.538471
7	bottle_3_blocks	bottle	-	3	32	3	1	2	2	True	0.10	0.0008	0.0001	15	None	0.5208	0.5318	1.301748	210.637655

Рисунок 4 - Итоговая таблица полученных результатов

Выводы

По итогам эксперимента наилучшее качество показали базовые сверточные модели (conv): лучшая конфигурация conv_a_k5 дала test_acc ≈ 0.816 (val_acc ≈ 0.816), а conv_b_small — test_acc ≈ 0.813. Residual-архитектуры в текущих настройках обучались заметно хуже (лучший результат res_5_blocks: test_acc ≈ 0.738), а bottleneck-варианты оказались наименее успешными (до test_acc ≈ 0.561). Это означает, что для CIFAR-10 в выбранном режиме обучения и ограничении на число эпох простая CNN оказалась наиболее эффективной, а residual/bottleneck-сетям, вероятно, требуется более тщательная настройка (например, больше эпох и/или иной режим регуляризации), чтобы раскрыть преимущество глубины.